

AEGIRA Planner

User Guide für die Phase-2-Spike-Plattform

Dieses Dokument erklärt in einfachem Deutsch, was die AEGIRA-Planner-Plattform heute leistet, wo welche Teile leben und wie du sie bedienst — ohne Tiefenwissen über Cloud-Infrastruktur. Wo technische Begriffe auftauchen, werden sie an Ort und Stelle erklärt.

Was	Wo
Live App (Demo)	https://zgpm.aegira.ai
Live API (Backend)	https://api.zgpm.aegira.ai
Source-Code (Versioniert)	github.com/exmachinai/agent_operating_model_planner
Source-of-Truth (Arbeitsstand)	Dropbox · 50_APPS/20_AGENT_OPERATING_MODEL_PLANNER/
Cloud-Stack	Azure Subscription, Sweden Central
Phase	Phase 2 — Spike (Demo-UI + Backend-Skelett)
Sockelkosten	≈ 80 €/Monat (Spike-Tier)

1 · Was ist die AEGIRA Planner Plattform?

AEGIRA Planner ist die App, die du nutzen wirst, um **Projekte methodisch nach ZGPM zu planen** und am Ende einen **Agent-Harness** als zip-Datei zu erhalten — das ist ein vorkonfiguriertes Paket, mit dem du in Claude Code mehrere KI-Agenten gleichzeitig arbeiten lassen kannst.

Die Plattform läuft auf Microsoft Azure, ist über das Web erreichbar, hat HTTPS-Verschlüsselung, eine Web Application Firewall (WAF) und einen Geo-Filter, der nur Anfragen aus der EU plus UK, Schweiz und USA durchlässt. Domain ist **zgpm.aegira.ai**.

Stand heute (29.05.2026) vs. später

Schon live (Phase 2 Spike)	Kommt in Phase 3+
• Web-UI mit AEGIRA-Branding	• Drei Demo-Komponenten (siehe Kapitel 2) • Speicher-Backend • Hohe Resilienz • Microsoft Entra ID

Faustregel: Wenn du heute auf zgpm.aegira.ai etwas anklickst und es etwas zeigt, ist es eine reine UI-Vorschau — die Daten kommen aus Beispiel-Konstanten im Code, nicht aus der echten Datenbank. Das macht jetzt nichts kaputt, aber speichert auch nichts.

2 · Die Live-Plattform bedienen

2.1 · Die Web-App: <https://zgpm.aegira.ai>

Öffne deinen Browser und gib die URL ein. Du landest auf einer Demo-Seite mit drei Buttons. Es gibt keinen Login, keine Daten — du kannst durchklicken ohne etwas zu beschädigen.

Was die drei Buttons machen

Button	Was passiert
AgentTrace zeigen	Blendet eine Karte ein, die zeigt wie später eine Agent-Aktivität visualisiert wird: Welcher Agent
Approval zeigen	Zeigt einen 'Human-in-the-Loop'-Approval-Dialog: ein Meilenstein steht zur Freigabe, der Meth
Sitzung sperren	Aktiviert den Lock-Screen — ein Vollbild-Overlay, das später nach 15 Min Inaktivität automatis

2.2 · Das Backend: <https://api.zgpm.aegira.ai>

Das Backend ist eine API, die später die ganze Logik enthält. Heute beantwortet es nur zwei Gesundheits-Endpunkte. Du brauchst dafür kein Tool, ein Browser reicht.

URL im Browser eintippen	Was kommt zurück
https://api.zgpm.aegira.ai/health	JSON mit {status: ok, service, env, version, now}
https://api.zgpm.aegira.ai/ready	JSON mit {status: ready, checks: {...}}

Wofür? Wenn du jemals den Verdacht hast, das Backend ist down: öffne /health im Browser. Kriegst du grünes JSON zurück, läuft alles. Kriegst du eine Fehlerseite, schau in Kapitel 5.

3 · Wo lebt was?

Die Plattform hat fünf Orte. Das ist die wichtigste Karte, die du verstehen musst — danach weißt du immer wo du hinmusst, wenn du etwas suchst.

Ort	Was lebt dort und wann gehst du hin
Dropbox-Ordner	Dein Arbeitsplatz. Hier liegt der gesamte Quellcode, alle Skripte, alle Dokumente. Wenn du etwas ändern willst, machst du das hier.
GitHub-Repo	Die offizielle Versionierung. Jede Änderung in der Dropbox wird über das Skript <code>_push-update.sh</code> in das GitHub-Repo übertragen.
Azure Portal	Die Cockpit-Ansicht der Live-Plattform. Hier siehst du, was Azure gerade tut: Container werden erstellt, Images werden geladen, etc.
Azure Container Registry	Das Lager der Docker-Images. Hier liegen die fertig gebauten App-Pakete (planner-app, observability-app, etc.).
~/Code-Folder auf deinem Mac	Temporäre Working-Copies. Wenn du <code>_push-update.sh</code> ausführst, klonst das Skript GitHub in deinen Code-Folder.

Wichtige IDs zum Nachschlagen

Was	ID / Pfad
Azure Subscription ID	23302507-c311-4f98-8af0-3061571960d4
Tenant ID (Entra)	611423c4-741d-40c9-8808-2271e4086ad2
Entra App ID (für SSO später)	1ffa4339-a156-45ea-ba3f-01cccc67f46d
Resource Group App	aegira-planner-prod
Resource Group Shared	aegira-shared-prod
Resource Group Observability	aegira-observability-prod

4 · Tägliche Operationen

Diese Befehle führst du im Terminal aus (auf macOS: Programme → Dienstprogramme → Terminal). Kopiere sie jeweils komplett, drücke Enter, fertig. Wenn etwas eine Authentifizierung verlangt, geht ein Browser-Tab auf — einfach klicken und genehmigen.

4.1 · Ist die Plattform online?

Schneller Check vom Terminal (oder im Browser einfach die URLs öffnen):

```
curl -s https://zgpm.aegira.ai/health
curl -s https://api.zgpm.aegira.ai/health
```

Beide sollten JSON mit **"status": "ok"** zurückgeben. Wenn nicht: Kapitel 5.

4.2 · Logs der Apps ansehen

Wenn etwas nicht stimmt, siehst du in den Logs was die App gerade macht. Im Terminal:

```
# Backend-Logs (letzte 30 Zeilen)
az containerapp logs show \
-g aegira-planner-prod \
-n ca-aegira-planner-api \
--type console --tail 30

# Frontend-Logs
az containerapp logs show \
-g aegira-planner-prod \
-n ca-aegira-planner-frontend \
--type console --tail 30
```

4.3 · Container neu starten

Wenn die App komische Dinge tut (selten), kannst du sie neu starten lassen. Das passiert in Sekunden:

```
# Aktuelle Revision rausfinden + neu starten
REV=$(az containerapp revision list \
-g aegira-planner-prod -n ca-aegira-planner-api \
--query "[-1].name" -o tsv)
az containerapp revision restart \
-g aegira-planner-prod -n ca-aegira-planner-api \
--revision "$REV"
```

4.4 · Code-Änderung deployen

Du hast in der Dropbox Code geändert (oder Claude hat). So bringst du es live:

Schritt 1 — Code zu GitHub pushen (Auto-merge passiert automatisch):

```
cd "/Users/mveil/Library/CloudStorage/Dropbox-exmachinAI/.../20_AGENT_OPERATING_MODEL_PLANNER"
bash _push-update.sh "Kurze Beschreibung der Änderung"
```

Schritt 2 — Wenn nur App-Code (Python/TypeScript) geändert wurde: Container-Images neu bauen + Apps umstellen:

```
bash _push-images.sh
```

Schritt 3 — Wenn auch Bicep (Infrastruktur) geändert wurde: zusätzlich Azure-Deploy:

```
bash _deploy-azure.sh  
# Subscription-ID eintippen, 2x Enter, dann 'yes' bei READY?
```

5 · Wenn etwas nicht funktioniert

5.1 · App lädt nicht / 502 / 503 / 504

Das sind Front-Door-Fehlermeldungen: 'Bad Gateway' (502), 'Service Unavailable' (503), 'Gateway Timeout' (504). Front Door erreicht die App nicht. Diagnose:

```
# Schritt 1 – Sind die Container Apps gesund?
for app in ca-aegira-planner-api ca-aegira-planner-frontend; do
echo "=== $app ==="
az containerapp revision list -g aegira-planner-prod -n $app \
--query "[?properties.active==\`true\`].{name:name, state:properties.runningState, \
health:properties.healthState, replicas:properties.replicas}" -o table
done
```

Beide sollten **Running + Healthy** mit mindestens 1 Replica zeigen.

Wenn Healthy, aber 502/504 trotzdem: Das ist meist temporär (Front Door braucht 1-2 Min um neue Origins zu erkennen). Warte 2 Min und versuche nochmal.

Wenn Unhealthy oder 0 Replicas: App ist abgestürzt. Logs lesen (4.2), Container neu starten (4.3). Falls hartnäckig: Claude fragen.

5.2 · Browser zeigt Zertifikatsfehler

Sollte nicht passieren — wir haben ein Let's-Encrypt-Cert via Azure Front Door. Wenn doch: wahrscheinlich abgelaufen oder DNS-Probleme. Verbinde dich mit Claude und lass die DNS-Records neu validieren.

5.3 · Kosten explodieren

Sollte nicht passieren, weil Budget-Alert bei 120 €/Mo (80% von 150 € Budget) eine Email schickt. Wenn die Email kommt: **Azure Portal** → **Cost Management** → **Cost Analysis** öffnen, schauen welche Resource teuer ist. Häufige Übeltäter: Cosmos RU/s (durch viel Traffic), Log Analytics (durch zu viel Logs).

6 · Was kostet das?

Wir laufen heute auf **Spike-Tier**: möglichst günstig, ausreichend für Demo und Phase-3-Entwicklung. Sobald die App produktiv genutzt wird, kannst du via ein-Zeilen-Bicep-Change auf Prod-Tier flippen.

Komponente	Spike (heute) — Prod (später)
Azure Front Door	Standard ≈ 35 € — Premium ≈ 280 €
Container Apps (Scale-to-zero)	≈ 15 € — Mit min 2 Replicas: ≈ 120 €
Cosmos DB Serverless	≈ 15 € — Multi-Region provisioned: ≈ 200 €
Container Registry Basic	≈ 5 € — Premium: ≈ 50 €
Storage LRS	≈ 3 € — RA-GRS: ≈ 5 €
Key Vault Standard	≈ 1 € — Premium: ≈ 5 €
Log Analytics 30d	≈ 10 € — 90d unlimited: ≈ 40 €
Summe (ohne LLM)	≈ 84 €/Monat — ≈ 700 €/Monat

Hinweis: Die Beträge sind Schätzungen anhand der Microsoft-Public-List-Prices. Tatsächlich zahlst du je nach Traffic mehr oder weniger. Bei Null-Nutzung eher 40 €/Monat (Spike-Tier), bei moderater Last eher 80–100 €. Für genaue Werte: Azure Portal → Cost Analysis.

Wo schaue ich die Kosten?

Portal-Weg: portal.azure.com → 'Cost Management + Billing' (linke Seitenleiste) → 'Cost Analysis'. Standardmäßig siehst du den aktuellen Monat, kannst nach Resource oder Resource Group filtern.

Email-Alarm: Liegt schon: bei 120 € (=80% des Budgets) bekommst du eine Email an exmachinai.ai@gmail.com. Diese Schwelle ist im Portal unter Cost Management → Budgets → aegira-planner-monthly konfigurierbar.

7 · Glossar — die Technik-Begriffe

Hier sind die Abkürzungen und Cloud-Begriffe, die im Code, in Befehlen und in Azure auftauchen. Mit dieser Liste verstehst du alles was ich heute geschrieben habe.

Begriff	Was es bedeutet
ACR (Azure Container Registry)	Privates Docker-Lager bei Azure. Hier liegen unsere fertig gebauten App-Images.
Bicep	Eine Sprache, in der unsere Infrastruktur beschrieben ist (Container Apps, Datenbank, Front Door)
Bytecamp	Unser Domain-Provider (aegira.ai liegt dort). Hier haben wir die DNS-Records angelegt, die zgpm.de auflösen.
CAE (Container Apps Environment)	Den 'Hof' bei Azure, in dem unsere zwei Apps (Backend + Frontend) als Container laufen.
Cold Start	Wenn ein Container von 0 auf 1 Replica hochfährt, dauert das ein paar Sekunden. Die erste Anfrage wird also etwas länger dauern.
Container	Ein verpacktes Software-Bundle inklusive Betriebssystem-Teil, das überall gleich läuft. Unser Backend und Frontend sind Container.
Cosmos DB	Microsofts NoSQL-Datenbank. Hier werden später die Projekte, Pläne und Sessions gespeichert.
FastAPI	Python-Framework, mit dem unser Backend gebaut ist. Die /health-Route ist eine FastAPI-Route.
Front Door (FD)	Microsofts 'Empfangstheke' im Internet. Sie nimmt alle Anfragen entgegen, filtert böse Anfragen (Virus, DDoS) und leitet sie an die Container Apps weiter.
Managed Identity (MI)	Eine Art 'Mitarbeiter-Ausweis' für unsere Apps. Damit beweist Backend gegenüber Cosmos, dass es eine legitime App ist.
Next.js	JavaScript-Framework, mit dem unser Frontend gebaut ist (React + Server-Rendering).
PR (Pull Request)	Ein Vorschlag auf GitHub, Code zu ändern. Wird von uns automatisch erstellt + gemergt.
Replica	Eine konkrete Kopie eines Containers, die gerade läuft. Heute haben wir 1 Replica pro App.
Revision	Eine bestimmte Version einer Container-App. Beim Image-Update wird eine neue Revision erstellt.
Spike-Tier vs Prod-Tier	Unsere zwei Kosten-Stufen. Spike = günstig für Entwicklung. Prod = teurer, aber redundant und skalierbar.
WAF (Web Application Firewall)	Schutz gegen typische Web-Angriffe (SQL-Injection, Bot-Traffic, Rate-Limit). Sitzt in Front Door.
ZGPM	ZielGerichtetes Projekt-Management — die methodische Grundlage des Planners. Aus PwC-Forschung.

8 · Was kommt als nächstes?

Heute haben wir die Plattform-Infrastruktur live gebracht. Hier ist der Plan was logisch folgt. Du musst nicht alles auf einmal — pick das was dir wichtig ist.

8.1 · Cosmos-Anbindung (Phase 3, Stage 2)

Das Backend bekommt seinen ersten echten Endpunkt: **POST /v1/projects**, der einen Projekt-Datensatz in Cosmos schreibt. Damit beweist du die End-to-End-Pipeline: Browser → Front Door → WAF → Backend → Managed Identity → Cosmos. Aufwand: ~90 Min.

8.2 · API-Design schärfen

Bevor du viel Backend-Code schreibst, lohnt sich ein sauberer OpenAPI-Spec: welche Endpoints gibt es, welche Felder, welche Fehler-Codes, welche Auth-Flows. Damit ist Phase 3 Stage 3 fast nur noch Codegen.

8.3 · Echtes Datenmodell (Phase 3, Stage 3)

Alle vier Cosmos-Container (projects, plans, sessions, audit) bekommen volle CRUD-Endpoints mit Pydantic-Schemas. Plus ein Auth-Stub gegen Entra ID. Mehrstündig.

8.4 · Lock-Screen mit Auto-Timer

Aktuell ist der Lock-Screen ein Demo-Button. Ziel: nach 15 Min Inaktivität automatisch sperren, nach 5 Min Inaktivität im Admin-Bereich, nach 30 Min im Hintergrund hart sperren. ~30 Min.

8.5 · Multi-Agent-Orchestrator (Phase 4)

Der Kern: fünf Planning-Agents (PMO, Phase-Architect, Activity-Engineer, Risk-Officer, Methodology-Guard) plus HITL-PM + Milestone-Executor. Diese laufen später in Claude Code als Subagents, gesteuert vom Backend. Großes Feature, eigene Phase.

Mein Tipp: Mach erst **8.2 (API-Design)**, dann **8.1 (Cosmos-Connect)**, dann **8.4 (Lock-Timer)**. Damit hast du in 1–2 Tagen einen funktionalen Stack, der echt Daten persistiert und die UX-Basis steht. Phase 4 ist dann ein eigenes Projekt.

Anhang · Die Skripte im Projekt-Ordner

Im Dropbox-Ordner siehst du Dateien, die mit Unterstrich beginnen (`_push-...`, `_deploy-...`). Das sind ausführbare Skripte. Hier was sie tun:

Skript	Was es tut · Wann du es nutzt
<code>_push-update.sh "Nachricht"</code>	Syncnt die Dropbox zu GitHub als Pull Request, der automatisch gemergt wird. Nutze: nach
<code>_deploy-azure.sh</code>	Liest den aktuellen Code aus <code>~/Code/</code> , baut die Azure-Infrastruktur per Bicep auf oder upda
<code>_push-images.sh</code>	Baut die Docker-Images für Backend + Frontend, pusht sie in die ACR, stellt die Container
<code>_HANDOVER_NEXT_MORNING</code>	Nutzen für den nächsten Arbeitstag — wo bin ich stehengeblieben, was muss ich morgen

Wichtig zur Reihenfolge

Wenn du sowohl Bicep als auch App-Code geändert hast, ist die Reihenfolge:

- 1) `_push-update.sh` — Code geht zu GitHub
- 2) `_deploy-azure.sh` — Azure-Infrastruktur wird aktualisiert
- 3) `_push-images.sh` — Images werden gebaut und Container Apps umgestellt

Dieses Dokument ist Teil der AEGIRA AI Trust Platform. Bei Fragen: Claude fragen oder exmachinai.ai@gmail.com schreiben.